Controllore di Demetra: controller_node.py

Documento di riferimento del nodo ROS 2 seg_inference_cmdvel_node_onnx (pacchetto seg_controller_onnx). Descrive cosa fa il nodo a ogni frame, con quali formule, quali valori si possono cambiare e dove. Tutti i valori sono quelli del codice al commit 8423c81.

1. Cosa fa il nodo

Il nodo riceve le immagini della camera, segmenta il percorso tra le file con una rete LR-ASPP (ONNX), estrae dalla maschera la mezzeria del corridoio con uno dei cinque estrattori (path finder), calcola l'errore laterale a una riga fissa dell'immagine, lo filtra e pubblica un comando di velocità /cmd_vel.

Camera → decodifica → resize 224×224 → LR-ASPP (ONNX) → maschera 0/1 → path finder → errore a y_ref → filtro errore → legge di sterzo → /cmd_vel. Sono inoltre pubblicati i topic di debug.

Topic

• Ingresso: /camera/camera/color/image_raw/compressed — sensor_msgs/CompressedImage; best effort, profondità 1 (si usa sempre l'ultimo frame).

• Uscita: /cmd_vel — geometry_msgs/Twist; si usano solo linear.x e angular.z; coda 20.

• /debug/segmentation/compressed — CompressedImage: maschera predetta con errore e stato.

• /debug/roi/compressed — CompressedImage: ROI dell'estrattore con strip, intervalli e rette.

• /debug/path/compressed — CompressedImage: immagine a colori originale con la linea stimata.

• /debug/text — std_msgs/String: riga di log con tempi, stato, errore e comandi, a ogni frame.

I topic /debug/*/compressed vengono pubblicati solo se enable_debug_images è true. /debug/text viene pubblicato sempre.

2. Avvio e parametri ROS

```
ros2 run seg_controller_onnx controller_node --ros-args \
    -p path_finder:=pf5 \
    -p enable_debug_images:=true \
    -p alpha_w:=0.30 \
    -p alpha_ref_fps:=30.0
```

Sono gli unici quattro parametri ROS. Tutto il resto (sezione 7) è una costante nel codice: per cambiarla si modifica il file e si ricompila il pacchetto.

• path_finder — default pf5: estrattore usato. Il valore viene letto una volta all'avvio.

• enable_debug_images — default true: genera e pubblica le tre immagini di debug. Con false si risparmiano circa 15 ms di CPU per frame (stima scritta nel commento del codice, non rimisurata).

• alpha_w — default 0.30: coefficiente del filtro sullo sterzo, definito alla frequenza alpha_ref_fps.

• alpha_ref_fps — default 30.0: frequenza di riferimento a cui valgono gli alpha dei filtri.

Valori di path_finder

• pf5 — Strip-IRLS — pf5.py.

• pf_net — EdgeNet — pf_net.py e modello models/rete_bordi.onnx.

• pf_ransac — RANSAC edges — pf_ransac.py.

• pf_centri — Center points — pf_centri.py.

• pf_histogram — Column histogram — pf_histogram.py.

Un valore diverso fa fallire l’avvio con ValueError. All’avvio il nodo chiama l’estrattore una volta su una maschera vuota, così la sessione ONNX di pf_net viene creata subito e non al primo frame.

3. Elaborazione di un frame (funzione callback)

1. Sottocampionamento. Si elabora un frame ogni process_every_n (ora 1, cioè tutti). Il commento accanto dice “30fps / 3 = 10 Hz”, ma il valore attivo è 1.

2. Tempo tra i frame. dt è il tempo dal frame elaborato precedente, limitato a [1/120 s, 0.25 s]. Al primo frame dt non esiste e i filtri usano l’alpha di riferimento.

3. Decodifica JPEG. Se fallisce, il nodo pubblica uno stop (v = w = 0) e salta il frame.

4. Resize a 224×224 con cv2.INTER_AREA. L’immagine 640×480 viene schiacciata, senza ritaglio.

5. Segmentazione (onnx_inference.run_inference_on_frame): conversione BGR → RGB, valori in [0, 1], normalizzazione ImageNet (media 0.485, 0.456, 0.406; deviazione 0.229, 0.224, 0.225); inferenza ONNX e argmax sui canali. Con il modello attuale (lraspp_224x224_30deg.onnx, 2 classi) la maschera vale 1 sul percorso e 0 altrove.

6. Estrattore. path_matrix, pf_debug = path_finder(pred_mask). Dal dizionario pf_debug il nodo usa status (ok, degraded o lost), confidence, centerline_model (la retta x = m·y + q) e i dati per le immagini di debug.

7. Errore laterale grezzo (sezione 4).

8. Filtro dell’errore (sezione 5).

9. Comando (sezione 6): stop se il filtro non dà un errore, altrimenti v e w dalla legge di sterzo. Il comando viene pubblicato prima delle immagini di debug, per non ritardarlo.

10. Debug. Si costruiscono e pubblicano le immagini (se abilitate) e la riga di testo.

4. Errore laterale (central_pixel_side_from_model)

La riga di controllo è:

```
y_ref = int(224 · ERROR_ROW_RATIO) = int(224 · 0.7) = 156
```

È la stessa riga usata in tutta la valutazione della tesi.

• Caso normale: l’estrattore fornisce la mezzeria x = m·y + q. Allora x_center = m·156 + q.

• Senza retta (estrattore lost): l’errore è None. Non c’è un ripiego sulla maschera di segmentazione: il filtro mantiene l’ultimo errore valido per hold_timeout = 0.5 s e poi il robot si ferma (sezione 5).

```
errore [px] = x_center − 224/2 = x_center − 112
```

• Errore > 0: la mezzeria è a destra del centro dell’immagine (side = "destra").

• Errore < 0: è a sinistra (side = "sinistra").

L’errore è in pixel della maschera 224×224, non in centimetri.

5. Filtro dell’errore (filter_error)

Il filtro ha uno stato: filtered_error, l’ultimo errore valido con il suo istante, e un contatore di salti consecutivi. I casi sono controllati in quest’ordine.

5.1 Estrattore lost

• Se c’è un errore valido più recente di hold_timeout = 0.5 s, il filtro mantiene l’ultimo errore filtrato (stato hold_lost). Il robot continua a sterzare come prima.

• Altrimenti azzera lo stato e restituisce None (stato lost_by_pf_status). Il nodo pubblica uno stop.

5.2 Errore grezzo assente (None) con estrattore non lost

Comportamento uguale: mantenimento per 0.5 s (stato hold), poi None (stato lost).

5.3 Errore presente

Gli alpha e la soglia sul salto dipendono dallo stato dell’estrattore:

• ok: ALPHA_ERR_OK = 0.20; soglia di salto 40 px.

• degraded: ALPHA_ERR_DEGRADED = 0.18; soglia di salto 38 px.

L’alpha viene corretto con il dt reale (sezione 5.4). Poi:

• Primo valore (o primo dopo uno stop): filtered_error = errore.

• Salto |errore − filtered_error| ≤ soglia: filtro esponenziale, filtered_error ← (1 − α)·filtered_error + α·errore.

• Salto oltre soglia, primo frame: è considerato un probabile errore isolato e pesato poco, filtered_error ← 0.8·filtered_error + 0.2·errore. Questo 0.2 non dipende da dt.

• Salto oltre soglia per il secondo frame consecutivo: è considerato un cambiamento reale e il filtro salta direttamente al nuovo valore, filtered_error = errore.

Dopo l’aggiornamento si salva l’errore valido con il suo istante.

Banda morta: se |filtered_error| < deadband_px = 4 px, l’errore passato alla legge di sterzo è 0. Lo stato interno del filtro però non viene azzerato.

Lo stato restituito è valid o valid_degraded e si legge nel log.

5.4 Alpha dipendente dal tempo (alpha_for_dt)

Gli alpha sono definiti alla frequenza alpha_ref_fps = 30 Hz. A ogni frame si usa:

```
α_eff = 1 − (1 − α_ref)^(dt · alpha_ref_fps)
```

Con dt = 1/30 s si ottiene esattamente α_ref. Se il nodo gira più lento (per esempio 18–20 Hz con le immagini di debug), α_eff cresce, così la costante di tempo in secondi resta uguale. Vale per il filtro sull’errore e per quello sullo sterzo.

6. Comando di velocità (compute_cmd) e arresto

6.1 Legge di sterzo

```
e_norm   = errore_filtrato / (224 / 2)
w_gain   = 0.6 + 0.4 · clip(confidence, 0, 1)
w_target = −kp_lat · w_gain · e_norm
w_target = clip(w_target, −w_max, +w_max)
w        = (1 − α_w)·w_precedente + α_w·w_target
v        = v_max
```

e_norm è in [−1, 1] per una mezzeria dentro l’immagine; w_gain va da 0.6 (confidence 0) a 1.0 (confidence 1); kp_lat = 1.2; α_w deriva da alpha_w corretto con dt.

• Segno: con errore positivo (corridoio a destra) w è negativo, cioè il robot gira a destra secondo la convenzione ROS (angular.z > 0 = sinistra).

• Confidence: riduce il guadagno fino al 40%. Con confidence bassa il robot sterza meno, ma non si ferma.

• Guadagno massimo: con confidence 1 e kp_lat = 1.2, w_target raggiunge w_max = 0.58 a e_norm ≈ 0.48, cioè circa 54 px di errore. È il “30° a ~54 px” del commento nel codice.

• Velocità: è sempre v_max. v_min è definita ma non viene usata: il robot non rallenta in curva né con stato degraded.

6.2 Quando il robot si ferma

Il nodo pubblica v = 0, w = 0 e azzera lo stato del filtro se:

• Il filtro restituisce None: per esempio l’estrattore è lost da più di 0.5 s, oppure non c’è mai stato un errore valido.

Il limite è solo di tempo reale (hold_timeout = 0.5 s dall’ultimo errore valido): non c’è un contatore di frame persi, quindi non dipende dal frame rate. Alla ripresa lo sterzo riparte da 0.

6.3 Da w all’angolo di sterzo

Il driver (demetra_serial_driver, driver_node.cpp) limita w a ±6 rad/s (max_angular_velocity_rad_s) e lo trasforma nell’angolo delle ruote sterzanti (kinematics.cpp, riga 60), con L = wheel_base = 0.4 m:

```
φ = atan2(L·w, 2·v)
φ_max = 30° (max_steering_angle_deg del driver)
```

A parità di w, più v è alta e meno il robot sterza. Per ottenere 30° servono:

• v_max = 0.07 m/s → w = 0.21 rad/s.

• v_max = 0.20 m/s → w = 0.58 rad/s.

• v_max = 0.30 m/s → w = 0.87 rad/s.

• v_max = 0.50 m/s → w = 1.44 rad/s.

I valori vengono dalla formula, non sono misurati sul robot. Una tabella più completa è nel commento del codice, intorno alla riga 366.

7. Costanti da cambiare nel codice

Tutte in seg_controller_onnx/controller_node.py.

• v_max, w_max (profilo): 0.20 m/s, 0.58 rad/s (“REALE 2”). Determinano velocità e sterzo massimo. Nel file sono commentati anche “REALE 1” (0.30 e 0.87) e “TEST” (0.07 e 0.21). Per le prime prove usa TEST. w_max va scelto con la tabella 6.3.

• kp_lat = 1.2: aggressività del rientro. Robot che oscilla → abbassare. Rientro lento → alzare a piccoli passi (0.1–0.2).

• deadband_px = 4 px: errore sotto cui non sterza. Piccole oscillazioni attorno al centro → alzare a 5–6.

• hold_timeout = 0.5 s: quanto tiene l’ultimo errore quando l’estrattore perde la linea. Robot che si ferma per buchi brevi → alzare poco. Robot che continua troppo dopo aver perso la linea → abbassare.

• ALPHA_ERR_OK = 0.20, ALPHA_ERR_DEGRADED = 0.18: filtro sull’errore. Errore filtrato che oscilla → abbassare. Risposta troppo lenta → alzare.

• Soglie di salto: 40 px (ok), 38 px (degraded). Oltre questo valore un salto è sospetto. Con un salto vero di più di 40 px il filtro impiega 2 frame a seguirlo.

• ALPHA_W (o parametro alpha_w) = 0.30: filtro sullo sterzo. Sterzo a scatti → abbassare. Sterzo in ritardo → alzare.

• ERROR_ROW_RATIO = 0.7 (riga 156): riga di controllo. Lasciare 0.7: la valutazione della tesi è fatta a questa riga e EdgeNet stima i bordi proprio alle righe 156 e 223.

• process_every_n = 1: elabora un frame ogni n. Lasciare 1.

• ENABLE_DEBUG_IMAGES = True: default del parametro ROS. Meglio usare il parametro ROS.

• Modello di segmentazione: models/lraspp_224x224_30deg.onnx (riga 191). Rete usata; da cambiare solo se si cambia il modello.

Definite ma non usate: v_min, alpha_min = 0.08, alpha_max = 0.45, jump_thr_min = 20 e jump_thr_max = 60. Cambiarle non ha effetto.

Parametri degli estrattori

Sono i valori di default delle funzioni path_finder nei rispettivi file e vanno cambiati lì.

• pf5.py (Strip-IRLS): min_row_ratio = 0.30, da regolare con la camera montata, in modo che la ROI contenga solo il percorso. La ROI viene estesa a un numero intero di strip e cambia a gradini: con 16 strip ogni valore tra 0.29 e 0.35 dà la ROI dalla riga 64.

• pf5.py: max_interval_gap = 40. Buchi o ombre più larghi spezzano il corridoio → alzare. Aggancia corridoi vicini → abbassare.

• pf5.py: n_strips = 16. Cambia anche l’altezza della ROI, quindi meglio non toccarlo.

• pf5.py: ok_conf_threshold = 0.60 e degraded_conf_threshold = 0.38: non calibrati.

• pf_ransac.py, pf_centri.py, pf_histogram.py: min_row_ratio = 0.30 con ROI dalla riga 67, senza estensione alle strip.

• pf_net.py (EdgeNet): non ha ROI in ingresso. Le soglie 0.5 su p_valid, 8 px di larghezza minima, 0.38 e 0.60 non sono calibrate per la sua confidence.

8. Leggere il log (/debug/text)

Esempio di riga in marcia (suddivisa qui su più righe):

```
Frame 1234 | callback_total=48.2 ms | decode=3.1 ms |
preprocess=0.6 ms | inference=21.0 ms |
path_finding=4.2 ms | cmd_publish=0.1 ms |
pf_status=ok | conf=0.82 | band_ratio=0.00 |
strip_ratio=0.00 | side=destra | raw_error=12.40 |
filtered_error=10.95px | filter=valid |
v=0.200 | w=-0.051 | dt=52.0 ms
```

I numeri sono un esempio di formato, non una misura.

• callback_total, inference, path_finding: tempi per frame. dt è il tempo reale tra due frame elaborati.

• pf_status, conf: stato e confidence dell’estrattore.

• raw_error, filtered_error: errore a y_ref prima e dopo il filtro, in pixel.

• filter: valid, valid_degraded, hold, hold_lost, lost o lost_by_pf_status.

• band_ratio, strip_ratio: venivano da estrattori vecchi. Gli estrattori attuali non li forniscono, quindi valgono sempre 0.00.

• Stop: la riga contiene STOP e lo stato del filtro (filter), e nel log ROS esce come warning.

Nel log ROS appare una riga ogni log_every_n = 5 frame. Sul topic arrivano tutte.

9. Procedura consigliata per le prove in campo

1. Montaggio e ROI. Con il robot fermo nel filare, guarda /debug/roi. La ROI deve iniziare sotto la zona dove il corridoio non si distingue più (fine fila, cielo, struttura della serra). Se serve, cambia min_row_ratio in pf5.py (e nelle baseline, se le usi).

2. Segmentazione. Su /debug/segmentation la maschera deve coprire il corridoio. Se non lo fa, il problema è il modello e non il controllore.

3. Prima marcia lenta. Profilo TEST (v_max = 0.07, w_max = 0.21), kp_lat = 1.2. Controlla che con il robot spostato a destra raw_error sia negativo e w positivo, e viceversa.

4. Aumenta la velocità passando a REALE 2 e poi a REALE 1, aggiornando w_max con la tabella 6.3.

5. Regolazione: oscillazioni → abbassa kp_lat, poi ALPHA_ERR_* o alpha_w; rientro lento → alza kp_lat; fermate frequenti → guarda pf_status: se è spesso lost, il problema è l'estrattore (ROI, gap, soglie). Se è ok ma filter è lost, controlla hold_timeout.

6. Confronto tra estrattori. Stesso percorso e stessi parametri del controllore, cambiando solo path_finder. Registra i topic (ros2 bag record /cmd_vel /debug/text e, se serve, le immagini di debug) per l'analisi della tesi.

10. Cose da sapere

• Provider ONNX: onnx_inference.py chiede CUDAExecutionProvider e poi CPUExecutionProvider. Il nodo scrive sempre "Inferenza forzata su CPU", ma il provider usato davvero è nella riga "Provider attivi sessione".

• Installazione dei modelli: il nodo cerca models/*.onnx accanto al file .py, ma setup.py non dichiara i file .onnx tra i dati da installare. Funziona con colcon build --symlink-install o lanciando dal sorgente. Con un build normale controlla al primo avvio che il log "Modello ONNX: …" punti a un file esistente.

• Nessuna memoria tra i frame negli estrattori: l'unica continuità temporale è il filtro del controllore (sezione 5).

• Tempi di Strip-IRLS: pf5.py ora usa la ROI estesa e 10 iterazioni IRLS. I tempi sulla Jetson non sono stati rimisurati: leggili da path_finding nel log.

11. Velocità angolare nei vari casi (simulazione)

Prove fatte con il controller vero (controller_node.py, pf5.py e gli altri estrattori), con ROS sostituito da un finto e un orologio simulato. Profilo attivo: v_max = 0.20 m/s, w_max = 0.58 rad/s, kp_lat = 1.2, alpha_w = 0.30, deadband 4 px. Lo sterzo è calcolato con la formula del driver. I valori sono w in rad/s e angolo delle ruote. Non sono misure sul robot.

11.1 Errore costante (valore a regime, 30 FPS)

errore [px]	confidence 0.0	confidence 0.5	confidence 1.0
2	0 (deadband)	0 (deadband)	0 (deadband)
5	0.03 / 1.8°	0.04 / 2.5°	0.05 / 3.1°
10	0.06 / 3.7°	0.09 / 4.9°	0.11 / 6.1°
20	0.13 / 7.3°	0.17 / 9.7°	0.21 / 12.1°
30	0.19 / 10.9°	0.26 / 14.4°	0.32 / 17.8°
40	0.26 / 14.4°	0.34 / 18.9°	0.43 / 23.2°
54	0.35 / 19.1°	0.46 / 24.8°	0.58 / 30.0°
70	0.45 / 24.2°	0.58 / 30.0°	0.58 / 30.0°
90	0.58 / 30.0°	0.58 / 30.0°	0.58 / 30.0°

Lo sterzo satura a 30° tra 54 e 90 px di errore, a seconda della confidence.

11.2 Gradino di errore (da 0 a E px, confidence 0.8)

gradino [px]	w finale / sterzo	90% a 30 FPS	90% a 10 FPS
20	0.20 / 11.2°	467 ms	500 ms
40	0.39 / 21.5°	467 ms	500 ms
54	0.53 / 28.0°	267 ms	400 ms
90	0.58 / 30.0°	267 ms	300 ms

I gradini grandi arrivano prima dei piccoli. Sopra circa 40 px il salto supera la soglia del filtro dell’errore e dopo 2 frame consecutivi viene accettato di colpo. Sotto i 40 px passa dal filtro esponenziale, che è più lento. A 10 FPS i tempi restano simili perché gli alpha sono corretti con il dt.

11.3 Outlier (errore vero 0)

• Un solo frame a +90 px: w arriva al massimo a 0.09, cioè 5.2° di sterzo. Il filtro lo assorbe.

• Due frame consecutivi a +90 px: il filtro li prende per un salto vero. w arriva a 0.32, cioè 17.8°, e torna a zero in circa 200 ms.

11.4 Maschere reali in sequenza (1604 frame, 13 sequenze, 30 FPS simulati)

estrattore	|w| med.	|w| p95	sterzo p95	|w| max	saturo	fermo
Strip-IRLS (pf5)	0.125	0.333	18.4°	0.527	0%	1.5%
RANSAC edges	0.134	0.365	20.1°	0.508	0%	0.1%
Column histogram	0.072	0.320	17.7°	0.579	0.7%	0.1%
Center points	0.148	0.357	19.6°	0.538	0%	0.1%
EdgeNet	0.140	0.359	19.7°	0.502	0%	0%

• Con tutti gli estrattori lo sterzo resta di solito sotto i 20° e satura quasi mai. L’unico caso è Column histogram, nello 0.7% dei frame, tutti su pomodori_maturi1.

• Il 1.5% di fermo di Strip-IRLS viene quasi tutto da pomodori_maturi1, dove il robot è fermo nell’8% dei frame perché la corsia manca per più di 0.5 s.

• La sequenza più impegnativa è pomodori_medi: con Strip-IRLS il 95° percentile dello sterzo arriva a 25°.

• Limiti: i comandi sono a ciclo aperto, perché le maschere vengono da video registrati e il comando non cambia la traiettoria. Il frame rate delle sequenze è assunto 30 FPS. pomodori2 è escluso perché i suoi frame

non sono consecutivi. Inerzia, motore e firmware non sono modellati. Sul robot i valori a regime saranno probabilmente più bassi.

11.5 Come impostare v_max e w_max per avere al massimo 30°

Il driver calcola l'angolo delle ruote φ = atan2(L·w, 2·v), con L = 0.4 m, e lo limita a 30°. Per arrivare a 30° serve w = 2·v·tan(30°)/L ≈ 2.89·v. Quindi basta impostare w_max = 2.89 · v_max.

v_max [m/s]	w_max per 30° [rad/s]	errore conf 1 [px]	errore conf 0 [px]
0.05	0.144	13	22
0.07	0.202	19	31
0.10	0.289	27	45
0.15	0.433	40	67
0.20	0.577	54	90
0.25	0.722	67	112
0.30	0.866	81	135
0.40	1.155	108	180
0.50	1.443	135	225

Con un w_max più alto della tabella lo sterzo resta comunque a 30°, perché il driver limita l'angolo, e la saturazione arriva con meno errore. Con un w_max più basso il robot non arriva mai a 30°. Gli errori in pixel valgono con kp_lat = 1.2 e immagine 224×224. Nelle colonne «errore» conf 1 vuol dire confidence 1 (w_gain = 1.0) e conf 0 vuol dire confidence 0 (w_gain = 0.6). I valori del codice (0.21, 0.58, 0.87, 1.44) sono questi arrotondati: la differenza è sotto 1°.